

Handle User Inputs and Forms

Making Requests from HTML Pages to PHP APIs
(Forms)

Learning Goals

By the end of this lesson, you will be able to:

1. Trace the five-step form submission workflow from the browser to PHP and back.
2. Intercept form submission and send `FormData` with `fetch()`.
3. Reuse the response-handling pattern from the previous lesson to display the result of a form submission.

Form Submission through the Front Controller

Frontend (`form_post_test.html`) ↔ Front Controller (`index.php`)

```
[HTML form] → [JavaScript] → POST /index.php/submit
      ↑               ↓
      └── update page ← JSON ← [index.php]
```

- As explained in the previous lesson, every API request enters through `index.php`.
- This form uses the registered `POST`

IMPORTANT

Five-Step Form Workflow

1. The user enters values and submits the form.
2. JavaScript intercepts the `submit` event.
3. JavaScript sends the fields in a POST request.
4. PHP reads the fields and returns JSON.
5. JavaScript displays the result or an error.

Before the Steps: Start the Application

```
php -S localhost:8000
```

Open http://localhost:8000/form_post_test.html.

Step 1: Define the Form Submission

- `action` identifies the PHP endpoint.
- `method` selects the HTTP request method.
- Each input's `name` becomes a submitted field name.

```
<form action="/index.php/submit" method="post">  
  Name: <input type="text" name="name"><br>  
  Email: <input type="email" name="email"><br>  
  <input type="submit" value="Submit">  
</form>
```

Without a `name`, an input is not included in the submitted form data.

Step 1 Reference: What Is an Endpoint?

- An API endpoint identifies a specific request that a server can handle.
- In this course, an endpoint is identified by an **HTTP method + URL path**.

```
POST /index.php/submit
```

The diagram shows the text 'POST /index.php/submit'. A horizontal line is drawn under the path '/index.php/submit', with a vertical line extending down from its left end to the label 'URL path'. Another horizontal line is drawn under the word 'POST', with a vertical line extending down from its left end to the label 'HTTP method'.

- Therefore, `POST /index.php/submit` is one endpoint.
- The same path with a different method, such as `GET /index.php/submit`, is treated as a different endpoint.
- A PHP file can handle one or more operations by inspecting the request and routing it to the correct code.

Step 1 Reference: Endpoint Used in This Lesson

```
POST /index.php/submit → supported
GET  /index.php/submit → 405 Method Not Allowed
POST /submit.php       → 404 Not Found
POST /submit.php/hello → 404 Not Found
```

- `index.php` accepts only paths registered in its router.
- The current Front Controller implements `GET` `/index.php/api` and `POST` `/index.php/submit`.

Step 2: Intercept Form Submission

- The `submit` event occurs when the form is submitted.
- `event.preventDefault()` keeps the browser on the current page.

```
document.querySelector('form').addEventListener('submit', async function (event) {  
    event.preventDefault();  
    // JavaScript now controls the submission.  
});
```

IMPORTANT

Step 3: Build and Send the POST Request

```
const form = event.target;  
const formData = new FormData(form);  
const response = await fetch(form.action, {  
  method: 'POST', body: formData  
});
```

- `event.target` is the submitted `<form>`.
- `new FormData(form)` collects all named fields.
- `form.action` resolves to `/index.php/submit`.
- `method: 'POST'` selects the POST method.
- `body: formData` sends the collected fields.

Reference: The Generated HTTP Request

JavaScript generates the following request:

```
POST /index.php/submit HTTP/1.1
Host: localhost:8000
Content-Type: multipart/form-data; boundary=...

--boundary
Content-Disposition: form-data; name="name"

Alice
--boundary
Content-Disposition: form-data; name="email"

alice@example.com
--boundary--
```

Step 4: Route, Validate, and Return JSON

Step 4.1: Accept Only POST Requests

```
case 'submit':  
  if ($method !== 'POST') {  
    sendError('Method not allowed for this endpoint', 405);  
  }
```

The endpoint returns HTTP status **405 Method Not Allowed** for other methods.

Step 4.2: Read the Form-Encoded Fields

This is an excerpt from the form-encoded (`else`) branch of the `submit` route in `index.php`.

```
if (strpos($contentType, 'application/json') !== false) {  
    ...  
} else { // force  
    $name = $_POST['name'] ?? '';  
    $email = $_POST['email'] ?? '';  
}
```

- This lesson sends `FormData`, so PHP follows the form-data branch.

Step 4.3: Build and Send the JSON Response

```
sendResponse([  
    'name' => $name,  
    'email' => $email,  
], 'Response from POST request');
```

IMPORTANT

Step 5: Display the Response or Error

We already learned how to check, parse, and display a JSON response. Here, we reuse that pattern after submitting the form.

```
try {
  if (!response.ok) {
    throw new Error(`HTTP error! status: ${response.status}`);
  }

  const data = await response.json();
  document.getElementById('result').textContent =
    JSON.stringify(data, null, 2);
} catch (error) {
  document.getElementById('result').textContent =
    'Error: ' + error.message;
}
```


Result Area Used in Step 5

```
<div id="result" class="result">  
  Submit the form to see the response...  
</div>
```

Complete JavaScript Code

```
document.querySelector('form').addEventListener('submit', async function (event) {
    event.preventDefault(); // Prevent the default form submission

    const form = event.target;
    const formData = new FormData(form);

    try {
        const response = await fetch(form.action, {
            method: 'POST',
            body: formData
        });

        if (!response.ok) {
            throw new Error(`HTTP error! status: ${response.status}`);
        }

        const data = await response.json();

        // Display the result
        document.getElementById('result').textContent =
            JSON.stringify(data, null, 2);
    } catch (error) {
        document.getElementById('result').textContent =
            'Error: ' + error.message;
    }
});
```

Complete Form Workflow

1. The user submits the HTML form.
2. JavaScript prevents normal page navigation.
3. `FormData` collects the inputs and `fetch()` sends the POST request.
4. `index.php` routes the endpoint, validates the method, reads the fields, and returns JSON.
5. JavaScript parses and displays the response or an error.

Key Takeaways

1. `action`, `method`, and `name` define form submission.
2. `preventDefault()` lets JavaScript control the submission.
3. `FormData` collects named form fields.
4. `fetch()` sends POST without leaving the page.
5. The `submit` route in `index.php` reads `FormData` through `$_POST`.